

Busos AI — Krytyczna ocena planu, wyjaśnienia i słowniczek

Dokument towarzyszący do `AI_PLAN.md` i `AI_research_potential.md`.

Napisany dla osoby, która robi taki projekt

pierwszy raz — tłumaczę każdy termin, każde narzędzie i każdy etap od zera.

Stan wiedzy: lipiec 2026, zweryfikowany przeciwko aktualnej dokumentacji narzędzi.

Jak czytać ten dokument

Dokument ma trzy części:

1. **CZĘŚĆ I — Krytyczna ocena planu.** Co jest dobre, co jest ryzykowne, co poprawić, jakie są lepsze/nowsze narzędzia. To jest ocena merytoryczna Twojego `AI_PLAN.md`.
2. **CZĘŚĆ II — Do czego służy każde narzędzie (dla laika).** Każde narzędzie z planu opisane po ludzku: co robi, po co jest, jaka jest jego rola w całości.

3. **CZĘŚĆ III — Wielki słownik.** Każdy termin branżowy, slangowy i techniczny, pogrupowany po etapach.

Jeśli gubisz się w terminie — skacz do CZĘŚCI III, tam wszystko jest wyjaśnione prostym językiem.

CZĘŚĆ I — Krytyczna ocena planu

1. Ocena ogólna: plan jest dobry, momentami wręcz bardzo dobry

Zacznę od rzeczy najważniejszej, bo łatwo się w krytyce zgubić: `AI_PLAN.md` **jest solidnym, dojrzałym dokumentem inżynierskim.** Jest znacznie lepszy niż `AI_research_potential.md`, na którym się opiera. Widać, że ktoś (Ty, albo Ty z asystą modelu) przeszedł od "burzy mózgów pełnej modnych haseł" do "konkretnego planu z etapami, kryteriami akceptacji i rejestrem ryzyk". To jest właściwy kierunek.

Co konkretnie jest zrobione **dobrze**:

- **Zasada "baseline first" (najpierw model bazowy).** Plan każe najpierw zbudować prosty

model (XGBoost), zmierzyć jego wynik, i dopiero potem sprawdzać, czy skomplikowany model grafowy (GNN) jest w ogóle lepszy. To jest **dokładnie** tak, jak robi się to profesjonalnie.

`AI_research_potential1.md` w kilku miejscach sugerował, że GNN "przewyższy klasyczne modele" — a plan słusznie to koryguje.

- **Uczciwość wobec wyników negatywnych.** Plan wprost mówi: jeśli GNN nie pobije XGBoosta, udokumentuj dlaczego — to też jest wartościowy wynik. To postawa naukowa, a nie marketingowa.
- **Świadomość ograniczeń sprzętu.** Plan liczy budżet pamięci VRAM, wprowadza `NeighborLoader`, ogranicza pamięć `r5py` do 12 GB. To realizm, którego brakuje wielu portfolio.
- **Nienaruszanie działającego pipeline'u (kroki 00–15).** AI działa na *wyjściach* starego systemu, nie przepisuje go. To ogranicza ryzyko, że popsujesz coś, co działa.
- **Kryteria akceptacji na każdym etapie.** Każda faza ma checklistę "co musi być spełnione". To zamienia mglisty pomysł w projekt, który da się faktycznie skończyć.

Poniżej — rzeczy do poprawy. Traktuj je jako "szlif", a nie jako "plan jest zły".

2. Poprawki merytoryczne (rzeczy, które naprawdę warto zmienić)

2.1 Instalacja PyTorch Geometric na AMD — plan jest zbyt pesymistyczny

Plan pisze (sekcja 1.2), że **nie istnieją gotowe "wheels" ROCm** dla `pyg-lib`, `torch-scatter`, `torch-sparse`, `torch-cluster`, i że musisz je budować ze źródeł, budżetując na to 1–2 dni.

To jest częściowo nieaktualne. Istnieje zewnętrzne repozytorium `pyg-rocm-build`, które udostępnia gotowe wheels dla ROCm dla tych właśnie rozszerzeń. Oficjalna dokumentacja PyG odsyła do niego wprost. To znaczy:

- **Najpierw** spróbuj zainstalować gotowe wheels z `pyg-rocm-build` dopasowane do Twojej wersji PyTorch + ROCm. To może zająć 15 minut zamiast 2 dni.
- **Dopiero jeśli** nie ma wheela pod dokładnie Twoją kombinację (PyTorch 2.9 + ROCm 7.2 + gfx1200) — wtedy budujesz ze źródeł.
- **Dobra wiadomość:** dla nowoczesnego PyG (2.5+) w wielu przypadkach `torch-scatter` / `torch-sparse` **nie są już wymagane** — PyG ma wbudowane natywne odpowiedniki (`scatter` z samego PyTorch). Sprawdź, czy

Twój model (GATv2) w ogóle ich potrzebuje, zanim zaczniesz walkę z kompilacją. Bardzo możliwe, że dla warstw uwagowych wystarczy samo `torch-geometric` + `torch`.

Poprawka do planu: zmień kolejność na "1. gotowe wheels z pyg-rocm-build → 2. sprawdź czy rozszerzenia są w ogóle potrzebne → 3. dopiero na końcu build ze źródeł". Oszczędzisz 1–2 dni.

2.2 API `city2graph` w planie jest zmyślane (halucynacja)

W planie (sekcja 4.2, Option A) jest kod:

```
from city2graph import Graph, Morphology
hetero_data =
Graph.gdf_to_pyg(nodes_gdf=...,
edges_gdf=..., ...)
```

Oraz w kryteriach akceptacji Fazy 0:

```
city2graph.Graph.gdf_to_pyg .
```

Taki interfejs nie istnieje. `city2graph` nie ma klasy `Graph` z metodą `gdf_to_pyg` ani klasy `Morphology`. Realne API to funkcje na poziomie modułu, np.:

```
import city2graph as c2g
# budowa grafu z danych
nodes, edges = c2g.morphological_graph(...)
# dla tkanki miejskiej
```

```
# lub contiguity_graph / proximity_graph /  
group_nodes  
# konwersja do PyTorch Geometric  
hetero = c2g.gdf_to_pyg(nodes_dict,  
edges_dict) # zwraca HeteroData
```

To ważne, bo gdybyś skopiował kod z planu 1:1, dostałbyś `ImportError` już na starcie i mógłbyś pomyśleć, że coś jest zepsute u Ciebie. **Nie jest — to plan ma błędny przykład.** Zanim napiszesz kod, otwórz aktualną dokumentację `city2graph.net` i przepisz przykłady stamtąd. To jest ogólna zasada: **kod w planie AI traktuj jako pseudokod ilustrujący intencję, nie jako gotowca do wklejenia.**

2.3 SHAP dla XGBoosta — dobrze; SHAP dla GNN — plan słusznie to koryguje

Plan poprawnie zauważa (Faza 5), że biblioteka `shap` **nie działa** z sieciami grafowymi (GNN), i że trzeba użyć `torch_geometric.explain` z `GNNExplainer` / `CaptumExplainer`. To dobra korekta względem `AI_research_potential.md`, który beztrósco mieszał SHAP z GNN.

Jedna uwaga praktyczna: `GNNExplainer` i `Integrated Gradients` **odpowiadają na różne pytania** i potrafią dawać różne wyniki. Nie traktuj ich jak "dwóch narzędzi do tego samego". Zdecyduj, co chcesz pokazać:

- "które **połączenia** (krawędzie) do POI napędzają cenę" → maski krawędzi z `GNNExplainer`,
- "które **cechy węzła** najbardziej podbijają predykcję" → `Integrated Gradients` / maski cech.

To jest istotne dla Fazy 5, bo Twój produkt końcowy (TOD Premium per przystanek) opiera się właśnie na tej interpretacji.

2.4 Podział train/test — dobrze pomyślany, ale pilnuj jednej rzeczy

Plan używa **przestrzennego podziału blokowego** (grupowanie po heksagonach H3 na poziomie 7), żeby uniknąć "spatial leakage" (wycieku przestrzennego). To jest bardzo dobre i wielu ludzi o tym zapomina.

Uwaga: przy małym mieście jak Kielce, po pogrupowaniu w duże heksy H3-7 możesz mieć **bardzo mało bloków**. Jeśli wyjdzie Ci np. 6–8 bloków, to podział 80/20 staje się kruchy (jeden nietypowy blok w teście potrafi wywrócić metryki). Rozważ **walidację krzyżową przestrzenną** (spatial CV) — kilka rotacji podziału — zamiast jednego sztywnego cięcia 80/20. Zmierzysz wtedy nie tylko wynik, ale i jego *niepewność*.

2.5 "Transport Deserts" i Deep Gravity — najbardziej ryzykowna część planu

Faza 6 (wykrywanie "pustyń transportowych" przez model grawitacyjny `scikit-mobility`) jest **konceptyjnie najslabszym ogniwem**, i to nie z winy planu, tylko z natury problemu:

- Model grawitacyjny generuje **syntetyczny** popyt (wirtualnych pasażerów). To jest *zgadywanie* przepływów, nie pomiar. Jeśli potem powiesz radnemu "tu jest pustynia transportowa", opierasz się na modelu, który sam w sobie jest hipotezą.
- `scikit-mobility` jest paczką starszą i wolniej rozwijaną niż reszta Twojego stacku. Sprawdź, czy w ogóle instaluje się czysto z Twoją wersją Pythona/NumPy, zanim oprzesz na niej całą fazę.
- **Sugestia:** potraktuj Fazę 6 jako *eksploracyjną* i mocno oznaczaj jej wyniki jako "modelowany szacunek popytu, do weryfikacji", a nie jako twardy fakt. Alternatywnie — dla portfolio silniejszym, uczciwszym sygnałem jest prosta, transparentna metryka dostępności (np. liczba mieszkańców w zasięgu X minut pieszo od przystanku o częstotliwości > Y, policzona `r5py`) niż egzotyczny model grawitacyjny, którego nikt nie zweryfikuje.

2.6 Faza 7 (propozycje nowych przystanków przez "wstawianie węzła do grafu") — uważaj na pułapkę logiczną

Pomysł: wstaw nowy przystanek do grafu, przepuść graf przez GNN jeszcze raz, zobacz jak zmieni się przewidywana cena. **Problem:** Twój GNN uczył się przewidywać cenę na podstawie *korelacji*, a nie *przyczynowości*. To, że model przewiduje wyższą cenę po dodaniu przystanku, **nie znaczy**, że dodanie przystanku *spowoduje* wzrost ceny. To klasyczny błąd "correlation $\neq$ causation" ubrany w sieć neuronową.

To nie znaczy "nie rób tego" — to znaczy "**nazywaj to ostrożnie**". Zamiast "dodanie przystanku podniesie ceny o 340 zł/m²" pisz "obszar wykazuje profil cech, który w naszych danych współwystępuje z wyższymi cenami — kandydat do dalszej analizy". Uczciwość w nazewnictwie to w tym projekcie Twój największy atut wobec konkurencji (patrz sekcja "no data fabrication" w Twoim własnym planie — trzymaj tę samą dyscyplinę w interpretacji).

2.7 Overengineering — kilka rzeczy jest "na wyrost" jak na MVP jednego miasta

Plan miejscami sięga po ciężką artylerię, która dla Kielc (jedno średnie miasto) jest przesadą. To nie błędy, ale świadome cięcia mogą Ci zaoszczędzić tygodnie:

- **Qdrant (baza wektorowa, Faza 8).** Dla jednego miasta z kilkuset przystankami wyszukiwanie podobieństwa działa równie dobrze na zwykłym NumPy/ `scikit-learn` (`NearestNeighbors`).

Qdrant ma sens dopiero, gdy masz setki tysięcy wektorów z całej Polski. **Sugestia:** w MVP pokaż podobieństwo bez Qdranta; Qdrant dołóż jako "amplifikator" dopiero przy skali krajowej.

- **PyTorch Lightning.** Świetny przy dużych, wielo-GPU treningach. Dla jednego małego grafu na jednej karcie to dodatkowa warstwa abstrakcji, która potrafi utrudnić debugowanie początkującemu. Czysty PyTorch (pętla treningowa na ~20 linii) jest tu całkowicie wystarczający i **bardziej edukacyjny** — zobaczysz, co się dzieje. To kwestia gustu, ale nie czuj presji, że "musisz" mieć Lightning.
- **FastAPI + osobny serwer inferencyjny.** Dla MVP wystarczy wyeksportować GeoJSON-y do pliku i wczytać je w dashboardie. Pełne API buduj, gdy faktycznie będziesz serwować zapytania "na żywo".

Zasada: MVP = najprostsza rzecz, która udowadnia tezę. Każdy dodatkowy komponent to dodatkowy punkt awarii i dodatkowy tydzień. Twój plan sam to deklaruje w zasadach — po prostu trzymaj się tego twardziej niż sam plan.

2.8 Wersje i "fakty" z

`AI_research_potential.md` — nie wszystkim ufaj

`AI_research_potential.md` jest pełen konkretnych, brzmiących wiarygodnie stwierdzeń, które są albo nieaktualne, albo zmyślane. Twój `AI_PLAN.md` już część z nich skorygował (Appendix A — bardzo dobra praktyka!). Kilka dodatkowych flag:

- Nazwy konkretnych publikacji ("BD-GNN, MDPI 2025", "A-SRGCNN 2025", "GNN-SWIM", "GNNUI arXiv maj 2025") — **zweryfikuj każdą z osobna, zanim ją zacytujesz**. Modele językowe notorycznie wymyślają wiarygodnie brzmiące tytuły prac naukowych. Jeśli chcesz się na coś powołać w portfolio/blogu, znajdź prawdziwy artykuł i przeczytaj abstrakt.
- "GRAPE do imputacji brakujących tagów OSM", "GraphMAE", "STGNN z ogranicznikami euklidesowymi" — to realne kierunki badawcze, ale dla Twojego MVP to rozpraszacze. Odlóż je świadomie (plan słusznie to robi).
- Ogólna zasada: **im bardziej konkretna liczba lub nazwa w `AI_research_potential.md`, tym bardziej ją sprawdź**. Ten dokument jest dobry jako *inspiracja kierunków*, a zły jako *źródło faktów*.

3. Sugestie lepszych / nowszych narzędzi

Poniżej alternatywy, uporządkowane od "zdecydowanie rozważ" do "opcjonalnie".

3.1 Zdecydowanie rozważ

LightGBM zamiast (lub obok) XGBoost jako

baseline. Plan wymienia XGBoost + CatBoost.

LightGBM jest zwykle **szybszy** w treningu przy podobnej jakości i świetnie radzi sobie z Twoim typem danych (dużo cech liczbowych, średni rozmiar).

Rekomendacja: uruchom wszystkie trzy (XGBoost, LightGBM, CatBoost) — to dosłownie kilka linii każdy, a porównanie trzech modeli w tabeli wygląda w

portfolio bardzo profesjonalnie. Aktualne badania

(2025–2026) potwierdzają zresztą to, co mówi Twój

plan: **modele drzewiaste wciąż wygrywają z**

sieciami neuronowymi na płaskich danych

tabelarycznych — więc dobry baseline drzewiasty to

nie formalność, to prawdopodobnie Twój najlepszy model.

Optuna do strojenia hiperparametrów.

Zamiast ręcznie zgadywać `max_depth`, `learning_rate` itd.,

Optuna przeszukuje je automatycznie i inteligentnie.

Kilkanaście linii kodu, a wynik baseline'u potrafi

podskoczyć. To też ładny sygnał dojrzałości w

portfolio.

3.2 Warto wiedzieć, że istnieją

PyG explain zamiast surowego Captum. Plan już to robi (dobrze). Tylko upewnij się, że używasz wysokopoziomowego API `torch_geometric.explain.Explainer`, a nie grzebiesz w Captum ręcznie — to pierwsze jest dużo prostsze.

Overture Maps zamiast/obok czystego OSM.

`AI_research_potential.md` wspomina o tym słusznie. Overture to "posprzątany" OSM z uzupełnionymi lukami (m.in. wysokości budynków). `city2graph` natywnie to wspiera. Dla MVP na Kielcach czysty OSM wystarczy, ale gdybyś miał dużo brakujących tagów — Overture jest realną poprawą jakości danych wejściowych **bez** budowania własnego modelu imputacji (który plan słusznie odkłada).

TabPFN / nowoczesne modele "foundation" dla tabel. To ciekawostka badawcza 2025–2026: modele, które "rozumieją" małe tabele bez treningu. Dla Kielc (mało danych na przystanek) mogłyby być zaskakująco dobre. To **eksperyment na później**, nie do MVP — ale wart wzmianki w sekcji "co dalej".

3.3 Rozważ ODŁOŻENIE (plan już to robi lub powinien)

- **Qdrant** — patrz 2.7. Dołóż przy skali krajowej.
- **PyTorch Geometric Temporal** (grafy czasowe) — plan słusznie odrzuca dla 32 GB RAM. Zgoda.

- **Neo4j** (baza grafowa) — plan słusznie odrzuca na rzecz PyG + Parquet dla treningu jednomaszynowego. Zgoda.
- **Reinforcement Learning / agent "przesuwający kursy"** (pomysł z `AI_research_potential.md`) — to fascynujące, ale to **osobny projekt na 6 miesięcy**, nie faza tego projektu. Bardzo dobrze, że nie ma go w `AI_PLAN.md`.

4. Ryzyka, których w planie brakuje

Twój rejestr ryzyk (sekcja 12) jest dobry, ale dopisałbym trzy pozycje:

Ryzyko	Dlaczego ważne	Co zrobić
Za mało transakcji RCN na przystanek	Przy medianie cen liczonej z kilku transakcji wynik jest bardzo niestabilny. Model uczy się szumu.	Ustal minimalny próg (np. ≥ 5 transakcji w promieniu), przystanki poniżej progu oznacz jako "brak"

Ryzyko	Dlaczego ważne	Co zrobić
		danych", nie zgaduj.
Data leakage przez cechy pochodne	Jeśli któraś cecha X jest policzona z tej samej wartości co Y (cena), model "oszukuje". Twój plan słusznie odcina <code>market_val</code> i wagi z kroku 15 — ale sprawdź każdą cechę pod tym kątem.	Audyt: dla każdej cechy zapytaj "czy mogłem to policzyć BEZ znajomości ceny?". Jeśli nie — wywal ją z X.
Nieporównywalność miast	Zasada "jedno miasto naraz" jest dobra, ale gdy w końcu porównasz Kielce z innym	Zanim uogólnisz, znormalizuj cenę wewnątrz miasta (np. percentyle) albo trenuj

Ryzyko	Dlaczego ważne	Co zrobić
	miastem, ceny są w różnych reżimach (Warszawa ≠ Elk).	z cechą "miasto".

5. Werdykt

Plan jest gotowy do realizacji po trzech drobnych poprawkach, które zrobisz w pół dnia:

1. Popraw sekcję instalacji PyG (najpierw gotowe wheels ROCm, sprawdź czy rozszerzenia w ogóle potrzebne) — **oszczędza 1-2 dni**.
2. Wyrzuć zmyślane API `city2graph` (`Graph.gdf_to_pyg`) i przepisz z aktualnej dokumentacji — **oszczędza frustrację na starcie**.
3. Oznacz Fazy 6 i 7 (pustynie, propozycje przystanków) jako **eksploracyjne** i pisz o nich językiem korelacji, nie przyczynowości — **chroni Twoją wiarygodność**.

Reszta planu (Fazy 0-5) jest solidna. Zacznij od niej, dowieź baseline (Faza 2) i GNN (Faza 4), a Fazy 6-8 traktuj jako "nice to have".

CZĘŚĆ II — Do czego służy każde narzędzie (wytłumaczone dla laika)

Tu tłumaczę każde narzędzie z planu tak, jakbyś nigdy wcześniej o nim nie słyszał. Format: **co to jest** → **po co Ci to** → **jaka jest jego rola w Twoim projekcie**.

Środowisko i sprzęt

ROCm

Co to jest: oprogramowanie od AMD, które pozwala Twojej karcie graficznej Radeon liczyć obliczenia sztucznej inteligencji. To odpowiednik CUDA (które jest dla kart NVIDIA).

Po co Ci to: karta graficzna liczy operacje na macierzach setki razy szybciej niż procesor. Bez ROCm Twój Radeon byłby do treningu bezużyteczny.

Rola w projekcie: fundament. Wszystko inne (PyTorch, GNN) stoi na ROCm. Jeśli ROCm nie widzi karty, nic dalej nie ruszy — dlatego to pierwszy krok (Faza 0).

PyTorch

Co to jest: najpopularniejsza biblioteka do budowania i trenowania sieci neuronowych.

Po co Ci to: to "silnik", w którym opisujesz model i który sam liczy, jak poprawiać jego wagi, żeby się uczył (tzw. propagacja wsteczna — patrz słowniczek).

Rola w projekcie: rdzeń całej części AI. GNN, którego trenujesz, jest napisany w PyTorch.

Budowa i trening modeli

XGBoost / LightGBM / CatBoost

Co to jest: trzy biblioteki tego samego typu — "gradient boosting na drzewach decyzyjnych". Budują tysiące małych drzewek decyzyjnych, gdzie każde kolejne poprawia błędy poprzednich.

Po co Ci to: to Twój **model bazowy (baseline)**.

Prosty, szybki, zwykle bardzo skuteczny na danych w formie tabeli (wiersze = przystanki, kolumny = cechy).

Rola w projekcie: Faza 2. Ustala poprzeczkę. GNN (Faza 4) musi tę poprzeczkę przeskoczyć, żeby udowodnić, że jest wart zachodu. Prawdopodobnie okaże się, że baseline jest bardzo trudny do pobicia — i to jest OK, tak mówi aktualna nauka.

PyTorch Geometric (PyG)

Co to jest: rozszerzenie PyTorch do sieci neuronowych **na grafach** (węzły + krawędzie), zamiast na tabelach czy obrazach.

Po co Ci to: miasto to naturalnie graf (przystanki = węzły, ścieżki między nimi = krawędzie). PyG pozwala nauczyć model, który "chodzi po grafie" i zbiera informacje od sąsiadów.

Rola w projekcie: Faza 3–4. To w PyG budujesz i trenujesz GNN (konkretnie GATv2).

GATv2 (Graph Attention Network v2)

Co to jest: konkretny typ sieci grafowej z "mechanizmem uwagi". Uwaga = model sam uczy się, którym sąsiadom przypisać większą wagę.

Po co Ci to: zamiast Ty ręcznie ustalać "galeria handlowa jest ważniejsza niż park" (jak w starym silniku v13.0 z wagami T0–T6), model **sam** wyciąga z danych, co realnie podbija ceny.

Rola w projekcie: to Twój główny model AI (Faza 4). Jego przewaga nad baseline'em to nie tyle lepszy wynik liczbowy, co zdolność uchwycenia **przestrzeni** — barier (rzeka, tory), które płaski model ignoruje.

city2graph

Co to jest: "klej" (2026) między danymi geograficznymi (GeoPandas/OSM/GTFS) a formatem grafowym PyG. Zamienia Twoje mapy w graf gotowy dla sieci neuronowej.

Po co Ci to: ręczne budowanie grafu heterogenicznego (`HeteroData`) jest żmudne i łatwo o błąd. city2graph robi to jednym wywołaniem.

Rola w projekcie: Faza 3, budowa grafu. **Uwaga:** przykłady API w Twoim planie są błędne — używaj funkcji z aktualnej dokumentacji city2graph.net (patrz CZĘŚĆ I, 2.2).

r5py

Co to jest: biblioteka Pythona, która oblicza **realny czas dojścia/dojazdu** między punktami, uwzględniając prawdziwe chodniki, ulice i rozkłady jazdy (GTFS). Pod spodem to szybki silnik R5 napisany w Javie.

Po co Ci to: odległość w linii prostej to kłamstwo — między domem a przystankiem może być rzeka bez mostu. r5py liczy, jak człowiek **naprawdę** dojdzie.

Rola w projekcie: Faza 3. Zamienia "5 minut w linii prostej" na "12 minut realnym chodnikiem". Te realne czasy stają się wagami krawędzi w grafie.

OSMnx

Co to jest: biblioteka zamieniająca dane OpenStreetMap w graf ulic i chodników (sieć drogową).

Po co Ci to: m.in. do "przyklejania" proponowanych przystanków do najbliższej ulicy (Faza 7) i jako źródło topologii sieci pieszej.

Rola w projekcie: pomocnicza, w Fazach 3 i 7.

H3 (Uber H3)

Co to jest: system dzielenia mapy na równe **sześciokąty** (heksagony) w wielu poziomach szczegółowości.

Po co Ci to: (1) do agregowania danych (ilu ludzi w tym heksie), (2) do uczciwego podziału train/test (grupujesz po heksach, żeby uniknąć wycieku), (3) do raportowania pustyni transportowych.

Rola w projekcie: przewija się przez cały plan jako "wspólna siatka" do liczenia i grupowania.

Interpretacja modelu (Explainable AI)

SHAP

Co to jest: metoda tłumacząca, **ile każda cecha dołożyła** do konkretnej predykcji. Oparta na teorii gier (wartości Shapleya).

Po co Ci to: żeby wyciągnąć z modelu bazowego liczbę typu "1 dodatkowy kurs/h dokłada X zł/m²" (Twój "TOD Premium").

Rola w projekcie: Faza 2, interpretacja XGBoosta.

Ważne: SHAP działa z drzewami (XGBoost), ale **nie** z GNN.

GNNExplainer / CaptumExplainer / Integrated Gradients

Co to jest: narzędzia do tłumaczenia sieci grafowych.

GNNExplainer mówi "które krawędzie/połączenia były ważne", Integrated Gradients mówi "które cechy węzła były ważne".

Po co Ci to: SHAP nie działa z GNN, więc to jest jego odpowiednik dla części grafowej.

Rola w projekcie: Faza 5. Zamiast jednej liczby dla całego miasta, dostajesz TOD Premium **osobno dla każdego przystanku** plus "które konkretne POI za to odpowiadają".

Statystyka przestrzenna

Moran's I (esda / libpysal)

Co to jest: miara, czy błędy Twojego modelu są **przestrzennie skupione**. Wysoki Moran's I na resztach = model przegapia coś przestrzennego.

Po co Ci to: to Twój "detektor", czy warto w ogóle iść w GNN. Jeśli błędy baseline'u są przestrzennie skupione (wysoki Moran's I), to znaczy "jest przestrzenna struktura, której płaski model nie łapie — GNN ma szansę pomóc".

Rola w projekcie: Faza 2 (brama decyzyjna: iść w GNN czy nie) i Faza 4 (dowód, że GNN faktycznie zmniejszył tę skupioność).

Generowanie popytu i propozycji

scikit-mobility (Deep Gravity)

Co to jest: biblioteka do modelowania **przepływów ludzi** — "ilu ludzi jedzie z A do B" — na podstawie tego, gdzie mieszkają i gdzie są cele (POI).

Po co Ci to: miasta kupują takie "macierze OD" za grube pieniądze od operatorów telekomów. Ty generujesz *syntetyczną* wersję.

Rola w projekcie: Faza 6, wykrywanie pustyń.

Uwaga: to najbardziej niepewna część (patrz CZĘŚĆ I, 2.5) — traktuj wyniki jako szacunek, nie fakt.

Warstwa serwująca (pokazanie wyników)

Qdrant

Co to jest: baza danych wyspecjalizowana w szybkim szukaniu "podobnych wektorów" (np. "znajdź okolice o profilu podobnym do mojej ulicy").

Po co Ci to: przy setkach tysięcy przystanków z całej Polski zwykle przeszukiwanie jest za wolne. Qdrant robi to w milisekundach + umie filtrować geograficznie.

Rola w projekcie: Faza 8. **Dla MVP na Kielcach to przerost formy** — zwykły `scikit-learn` wystarczy (patrz CZĘŚĆ I, 2.7).

FastAPI

Co to jest: biblioteka do budowy szybkiego API (serwera, który odpowiada na zapytania z przeglądarki/dashboardu).

Po co Ci to: żeby frontend (dashboard w Next.js) mógł poprosić "daj mi dane dla Kielc" i dostać je na żywo.

Rola w projekcie: Faza 8. Dla MVP możesz zacząć od eksportu plików GeoJSON i dołożyć API później.

GeoJSON

Co to jest: format pliku zapisujący obiekty geograficzne (punkty, obszary) z ich właściwościami. Czytają go wszystkie mapy webowe.

Po co Ci to: to "waluta wymiany" między Twoim AI a mapą w dashboardzie.

Rola w projekcie: format wyjściowy Faz 5–7 (pustynie, propozycje, TOD Premium na mapie).

CZĘŚĆ III — Wielki słownik

Terminy pogrupowane po obszarach. W obrębie grupy — od najważniejszych. Jeśli szukasz konkretnego słowa, użyj Ctrl+F.

A. Podstawowe pojęcia uczenia maszynowego

Model — program, który uczy się wzorca z danych, zamiast być zaprogramowany regułami. Tu: uczy się związku "cechy okolicy → cena mieszkania".

Cecha (feature), oznaczana X — pojedyncza informacja wejściowa o obiekcie. U Ciebie np. `transit_freq` (ile autobusów/h), `poi_count_health` (ile przychodni w pobliżu). Model dostaje zestaw cech i na ich podstawie zgaduje odpowiedź.

Zmienna celu (target), oznaczana Y — to, co model ma przewidzieć. U Ciebie: `price_m2` (cena za metr kwadratowy). To jest "prawidłowa odpowiedź", z którą model porównuje swoje zgadywanie.

Regresja — typ zadania, gdzie przewidujesz **liczbę** (cena), a nie kategorię. (Przewidywanie kategorii to "klasyfikacja".)

Uczenie nadzorowane (supervised learning) — uczenie, gdzie masz "prawidłowe odpowiedzi" (ceny z RCN) i model uczy się je odtwarzać. Przeciwnieństwo: nienadzorowane (bez odpowiedzi, model sam szuka struktury).

Trening / trenowanie — proces, w którym model wielokrotnie ogląda dane, zgaduje, sprawdza błąd i

poprawia się.

Zbiór treningowy / testowy (train/test split) —

dzielisz dane na część, na której model się uczy (train, np. 80%), i część, której nigdy nie widział podczas nauki (test, 20%), żeby uczciwie sprawdzić, czy się nauczył, a nie "wykuł na pamięć".

Wyciek danych (data leakage) — sytuacja, gdy do

modelu przypadkiem trafia informacja, której "nie powinien mieć" (np. cecha policzona z ceny). Model wtedy "oszukuje" i wygląda genialnie na teście, a w praktyce jest bezużyteczny. Twój plan mocno się przed tym broni.

Wyciek przestrzenny (spatial leakage) — odmiana

wycieku: gdy punkt treningowy i testowy leżą tak blisko, że są prawie tym samym miejscem. Model "podgląda" odpowiedź przez sąsiedztwo. Lekarstwo: dzielić train/test po blokach przestrzennych (heksy H3).

Przeuczenie (overfitting) — model "wykuł na

pamięć" dane treningowe zamiast zrozumieć wzorzec. Świetny na treningu, słaby na nowych danych. Objaw: duża różnica wyniku train vs test.

Niedouczenie (underfitting) — odwrotność: model

za prosty, nie złapał nawet oczywistego wzorca.

Hiperparametr — ustawienie modelu, które Ty

wybierasz **przed** treningiem (np. `max_depth` ,

`learning_rate`). W odróżnieniu od wag, których model uczy się sam.

Strojenie hiperparametrów (hyperparameter tuning) — szukanie najlepszych ustawień, np. ręcznie albo automatycznie (Optuna).

Walidacja krzyżowa (cross-validation) — zamiast jednego podziału train/test robisz kilka rotacji i uśredniasz. Daje pewniejszą ocenę i informację o niepewności.

Baseline (model bazowy) — najprostszy sensowny model, który ustala poprzeczkę. Każdy skomplikowany model musi go pobić, inaczej nie ma sensu.

B. Metryki (jak mierzymy jakość)

MAE (Mean Absolute Error, średni błąd bezwzględny) — średnio o ile złotych model się myli. $MAE = 500$ zł znaczy "przeciętnie chybiam o 500 zł/m²". Im mniej, tym lepiej. Łatwe do interpretacji.

R² (współczynnik determinacji) — jaki % zmienności ceny model wyjaśnia. $R^2 = 0$ to "model bezużyteczny, tyle samo co zgadywanie średniej", $R^2 = 1$ to "idealnie". Twój plan chce $R^2 > 0,5$ na teście.

MSE (Mean Squared Error, błąd średniokwadratowy) — jak MAE, ale karze duże

pomyłki mocniej (podnosi błąd do kwadratu).
Używany jako "funkcja straty" w treningu.

Funkcja straty (loss function) — liczba, którą model podczas treningu **minimalizuje**. To jego "kompas" — mówi, jak bardzo się myli. U Ciebie: MSE.

Reszty (residuals) — różnice między prawdą a predykcją ($Y - \text{predykcja}$) dla każdego punktu. Analiza reszt zdradza, gdzie i jak model się myli.

C. Grafy i sieci grafowe

Graf — struktura z **węzłów** (nodes) i **krawędzi** (edges) je łączących. U Ciebie: węzły = przystanki i POI, krawędzie = ścieżki/czasy dojścia.

Węzeł (node / vertex) — punkt w grafie. Przystanek, budynek, POI.

Krawędź (edge) — połączenie między węzłami. Może mieć **cechy** (np. czas przejścia).

Cecha krawędzi (edge attribute / edge_attr) — liczba(y) przypisane do połączenia, np.

[czas_przejścia]. Kluczowe w urbanistyce: ważne jest nie tylko *co* jest w węźle, ale *jak trudno* tam dojść.

Graf heterogeniczny (HeteroData) — graf z **różnymi typami** węzłów i krawędzi (np. typ "przystanek", typ "POI", krawędź "dochodzi_do"). Twój graf jest heterogeniczny.

GNN (Graph Neural Network, grafowa sieć neuronowa) — sieć, która uczy się na grafie, przekazując informacje między sąsiadami.

Message passing (przekazywanie wiadomości) — mechanizm działania GNN: każdy węzeł "zbiera" informacje od sąsiadów, aktualizuje swój stan, i tak warstwa po warstwie. Po 2 warstwach węzeł "wie" o sąsiadach i sąsiadach sąsiadów.

Mechanizm uwagi (attention) — technika, w której model uczy się, **którym** sąsiadom/cechom przypisać większą wagę, zamiast traktować wszystkich równo. Serce GATv2.

GATv2 / GAT (Graph Attention Network) — sieć grafowa z mechanizmem uwagi. "v2" to poprawiona wersja z lepiej działającą uwagą.

GraphSAGE — inny popularny typ GNN (próbkuje sąsiadów i agreguje). W planie wymieniony jako alternatywa do porównania.

Głowa uwagi (attention head) — jedna "perspektywa" mechanizmu uwagi. Kilka głów (np. 4) uczy się różnych wzorców naraz, potem się je łączy.

Node embedding (osadzenie węzła / wektor osadzeń) — po treningu każdy węzeł ma przypisany wektor liczb (np. 64 liczby), który "streszcza" jego sytuację w grafie. Podobne miejsca mają podobne

wektory. To pozwala szukać "okolic podobnych do mojej".

Eksplzja sąsiedztwa (neighborhood explosion) —

problem: przy 2–3 skokach po grafie liczba sąsiadów rośnie lawinowo i zapycha pamięć karty (VRAM).

Rozwiązanie: NeighborLoader.

NeighborLoader — narzędzie PyG, które zamiast

całego grafu bierze małe **podgrafy** (węzeł + próbka sąsiadów) i tylko to wrzuca na kartę. Ratuje VRAM. W

Twoim planie obowiązkowy przy 16 GB.

Węzeł izolowany (isolated node) — węzeł bez

żadnej krawędzi. GNN nie umie się o nim uczyć (nie ma od kogo zbierać informacji). Plan sprawdza, że takich nie ma.

Propagacja wsteczna (backpropagation) —

mechanizm, dzięki któremu sieć wie, jak poprawić każdą swoją wagę, żeby zmniejszyć błąd. "Cofa się" od błędu do wag.

Epoka (epoch) — jedno pełne przejście modelu przez

wszystkie dane treningowe. Trening to zwykle wiele epok (u Ciebie do 200).

Early stopping (wczesne zatrzymanie) —

przerwanie treningu, gdy model przestaje się poprawiać, żeby nie przeuczyć. "Patience=20" = "jeśli przez 20 epok nie ma poprawy, stop".

Dropout — technika: podczas treningu losowo "wyłączasz" część sieci, co zapobiega przeuczeniu.
0,3 = wyłączasz 30%.

D. Dane przestrzenne (GIS)

GIS (Geographic Information System) — ogólnie: praca z danymi, które mają lokalizację na mapie.

GeoPandas — Pandas (tabele) + geometria.
Podstawowe narzędzie do danych przestrzennych w Pythonie.

GeoDataFrame (gdf) — tabela, gdzie jedna kolumna to geometria (punkt/linia/obszar).

Geometria — kształt obiektu: punkt (przystanek), linia (ulica), poligon/wielokąt (budynek, działka).

Centroid — środek ciężkości obszaru. Np. środek heksagonu albo budynku.

WGS84 (EPSG:4326) — układ współrzędnych "z GPS": szerokość/długość geograficzna (lat/lon).
Dobry do wyświetlania, zły do mierzenia odległości w metrach.

EPSG:2180 — polski układ współrzędnych w **metrach**. Używasz go, gdy chcesz liczyć odległości w metrach bez zniekształceń. Stąd kolumny `x_2180`, `y_2180`.

Odległość euklidesowa — odległość w linii prostej ("jak lecą ptaki"). Szybka, ale w mieście często myląca (nie uwzględnia rzek, murów).

Odległość sieciowa (network distance) — realna odległość po chodnikach/ulicach ("jak chodzą ludzie"). Liczona przez r5py/OSMnx. Prawdziwsza, wolniejsza.

Izochrona — obszar, do którego dojdiesz w X minut z danego punktu. "Co jest w zasięgu 15 min pieszo od przystanku".

Macierz czasów podróży (Travel Time Matrix, TTM) — tabela "z każdego punktu do każdego punktu: ile minut". Produkt r5py.

Bufor (buffer) — obszar w promieniu X metrów wokół punktu. "Wszystko w 500 m od przystanku".

POI (Point of Interest) — obiekt istotny: szpital, szkoła, sklep, park. U Ciebie pogrupowane w 6 domen (zdrowie, edukacja, handel, rekreacja, administracja, transport).

Heksagon / H3 — patrz CZĘŚĆ II. Sześciokątna komórka siatki. Zaleta nad kwadratami: wszyscy sąsiedzi w równej odległości.

Rozdzielczość H3 (H3 resolution) — poziom szczegółowości siatki. Wyższa liczba = mniejsze hekсы. `h3_res9` (drobne) do agregacji cech, `h3_res7` (grube) do podziału train/test.

Tesselacja (tessellation) — pokrycie mapy siatką (np. heksagonów) bez dziur i nakładek.

E. Źródła danych w projekcie

GTFS (General Transit Feed Specification) — światowy standard rozkładów jazdy (linie, przystanki, godziny). Twoje źródło `transit_freq` i `span` godzinowy.

OSM (OpenStreetMap) — otwarta mapa świata tworzona społecznościowo. Źródło budynków, ulic, POI.

Overture Maps (OMF) — "posprzątany" konkurent OSM (Meta, Amazon, Microsoft, TomTom), z uzupełnionymi lukami. Wspierany przez city2graph.

PBF (.osm.pbf) — skompresowany format pliku z danymi OSM. Wrzucasz go do r5py.

RCN (Rejestr Cen Nieruchomości) — polski rejestr transakcji nieruchomości (GUGiK). Twoje **ground truth** (prawdziwe ceny), z których uczy się model.

Ground truth — "prawda", z którą porównujesz predykcje. U Ciebie: prawdziwe ceny z RCN.

GUGiK — Główny Urząd Geodezji i Kartografii. Wystawia RCN.

GUS / NSP 2021 — Główny Urząd Statystyczny / Narodowy Spis Powszechny. Źródło siatki demograficznej (ilu ludzi mieszka gdzie).

WFS (Web Feature Service) — internetowy protokół pobierania danych geograficznych z serwerów urzędowych. Stąd pobierasz RCN. Bywa niestabilny — stąd skrypty naprawcze 08/09.

TERYT — polski system kodów terytorialnych (identyfikatory gmin/powiatów). Do cache'owania po obszarach.

hstore / all_tags — sposób trzymania "wszystkich tagów OSM" jednego obiektu w jednej kolumnie jako pary klucz-wartość (np. `building=yes, levels=4`).

GeoPackage (.gpkg) — format pliku bazy danych przestrzennej (jak plik SQLite z geometrią). Twoje `stops.gpkg`, `infrastructure.gpkg`.

Parquet (.parquet) — bardzo wydajny format tabel kolumnowych. Szybki do wczytywania dużych macierzy cech. Twój format wyjściowy dla AI.

F. Pojęcia z Twojego istniejącego silnika (v13.0)

TOD (Transit-Oriented Development) — rozwój miasta zorientowany na transport publiczny; idea, że dobra komunikacja podnosi wartość i jakość okolicy.

TOD Premium — "ile wart jest transport w cenie mieszkania" — np. ile zł/m² dokłada 1 dodatkowy kurs/h. Twój flagowy wynik.

Model Huffa / grawitacja miejska — model mówiący, że atrakcyjność miejsca "przyciąga" ludzi tym słabiej, im dalej ono leży (jak grawitacja). Twój stary silnik liczy to ręcznie; AI ma to zastąpić uczeniem z danych.

Tier (T0–T6) — Twój ręczny system wag POI (T0 = megahub = 5 mln pkt, T6 = park/kościół = 100 pkt). Krytyka w `AI_research_potential.md` : to ludzkie uprzedzenie wstrzyknięte na sztywno. Dlatego dane dla AI eksportujesz **bez** tierów (surowe liczniki).

Entropia Shannona — miara "różnorodności" funkcji w okolicy (mixed-use). Wysoka entropia = i sklepy, i biura, i mieszkania = "tętniące życiem".

Z-Score (standaryzacja) — przeliczenie wartości na "ile odchyień od średniej". Ujednolica cechy w różnych skalach. Twój stary silnik tego używa; dla AI eksportujesz surowe wartości, żeby model sam zdecydował o skali.

IQR (Interquartile Range, rozstęp międzykwartyłowy) — zakres "środkowych 50%" danych. Do odcinania wartości absurdalnych (outlierów) przy liczeniu mediany cen.

MAD (Median Absolute Deviation) — odporna miara rozrzutu; "6-sigma MAD" wykrywa ekstremalne błędy (np. "teleportujące się" stacje PKP).

Outlier (wartość odstająca) — punkt drastycznie odbiegający od reszty. Może być błędem (do usunięcia) albo rzadkim, ważnym sygnałem (do zachowania). Rozróżnienie bywa trudne — patrz ostrzeżenie o "pomaganiu modelowi" w planie.

Spatial Dissolve — sklejanie rozczłonkowanych obiektów OSM (szpital z 15 pawilonów) w jeden, żeby nie liczyć ich 15 razy.

Agglomerative Clustering (klastrowanie aglomeracyjne) — algorytm łączący bliskie punkty w grupy. Twój stary silnik scala przystanki progami 150 m/100 m. Krytyka: to niszczy wariancję potrzebną GNN — dla AI tego nie robisz.

G. Interpretowalność i etyka

XAI (Explainable AI, wyjaśnialna AI) — dziedzina o tłumaczeniu, **dlaczego** model podjął decyzję. Kluczowa, gdy wynik idzie "na stół radnego".

SHAP / wartości Shapleya — patrz CZĘŚĆ II. Ile każda cecha dołożyła do predykcji.

Mechanistic interpretability (interpretowalność mechanistyczna) — głębsze zagłębienie "do środka"

modelu, do jego wag/aktywacji, by zrozumieć jego wewnętrzne działanie. `AI_research_potential.md` wspomina o Twoim doświadczeniu w tym obszarze przy modelach językowych.

Aktywacje — wartości "płynące" przez sieć podczas liczenia. Ich analiza pomaga zrozumieć, co model "myśli".

EU AI Act — unijne rozporządzenie o AI; wymaga m.in. przejrzystości modeli w zastosowaniach istotnych. Stąd nacisk planu na wyjaśnialność.

Korelacja vs przyczynowość (correlation vs causation) — fundamentalne rozróżnienie: to, że dwie rzeczy występują razem (korelacja), nie znaczy, że jedna powoduje drugą (przyczynowość). Twój model widzi korelacje. Ostrożność w nazywaniu wyników (Faza 7) chroni Cię przed nadużyciem.

H. Sprzęt i środowisko

GPU / VRAM — karta graficzna / jej pamięć. Trening AI dzieje się na GPU; VRAM (u Ciebie 16 GB) to twardy limit rozmiaru tego, co naraz wrzucisz.

RDNA 4 / gfx1200 — architektura / kryptonim Twojej karty Radeon RX 9060 XT. ROCm musi ją rozpoznać.

CUDA — odpowiednik ROCm dla NVIDIA.

`torch.cuda.is_available()` w PyTorch działa też

dla AMD przez ROCm (mylące, ale tak jest).

wheel (.whl) — gotowy, prekompilowany pakiet Pythona. "Gotowe wheels" = instalacja bez kompilacji (szybko). "Build ze źródeł" = samodzielna kompilacja (wolno, ryzykownie).

Kompilacja ze źródeł (build from source) — samodzielne zbudowanie pakietu z kodu źródłowego, gdy nie ma gotowego wheela pod Twój sprzęt. Bywa uciążliwe.

Docker — narzędzie do uruchamiania programów w izolowanym "pudełku" z gotowym środowiskiem. Plan proponuje obraz `rocm/pytorch` jako plan B, gdy instalacja się nie uda.

JDK 21 (Java Development Kit) — Java potrzebna przez r5py (silnik R5 jest w Javie). Musi być w wersji 21+.

Conda / pip — dwa menedżery pakietów Pythona. Conda bywa wygodniejsza dla trudnych zależności (np. r5py + Java).

venv (środowisko wirtualne) — izolowany "worek" na pakiety jednego projektu, żeby nie mieszać z systemem.

OOM (Out Of Memory) — "zabrakło pamięci". Częsty błąd przy zbyt dużym grafie na GPU/RAM. Cały wątek

NeighborLoader/12 GB w planie jest po to, żeby go uniknąć.

psutil — biblioteka do mierzenia zużycia pamięci/CPU. Plan używa jej do pilnowania, czy r5py nie przekracza 20 GB RAM.

I. Architektura i inżynieria

Pipeline (potok) — ciąg kroków przetwarzania danych, jeden po drugim (Twoje kroki 00–15).

ETL (Extract, Transform, Load) — wzorzec: wyciągnij dane → przekształć → załaduj. Twój pipeline to "Spatial ETL".

Orchestrator (orkiestrator) — program zarządzający kolejnością kroków pipeline'u (u Ciebie `orchestrator.py`).

Feature matrix / feature tensor (macierz/tensor cech) — tabela liczb podawana modelowi: wiersze = obiekty (przystanki), kolumny = cechy.

Tensor — wielowymiarowa tablica liczb; podstawowa "jednostka danych" w PyTorch.

Embedding — patrz "node embedding". Ogólnie: zamiana czegoś złożonego na wektor liczb, w którym podobne rzeczy są blisko siebie.

Baza wektorowa (vector database) — baza wyspecjalizowana w szukaniu podobnych wektorów (Qdrant). Patrz CZĘŚĆ II.

KNN / k-NN (k Nearest Neighbors) — "k najbliższych sąsiadów". Metoda znajdowania najbardziej podobnych/najbliższych punktów. Używana i do wag przestrzennych (Moran's I), i do wyszukiwania podobieństwa.

API (Application Programming Interface) — "gniazdko", przez które jeden program prosi drugi o dane/usługę. FastAPI buduje takie gniazdko.

Endpoint — konkretny adres w API, np.
`/api/v1/deserts/kielce` = "daj pustynie dla Kielc".

MVP (Minimum Viable Product) — najmniejsza wersja projektu, która działa i udowadnia tezę. U Ciebie: Kielce, Fazy 0–4/5.

Inference (inferencja) — użycie **wytrenowanego** modelu do przewidywania na nowych danych (bez dalszego uczenia). Np. "przepuść graf przez gotowy GNN i odczytaj predykcje".

Checkpoint — zapisany stan wytrenowanego modelu (plik `.pt`), żeby nie trenować od nowa.

Snapshot — "migawka" stanu w czasie; w kontekście grafów czasowych = graf w jednym momencie (np. rozkład o 8:00).

Na koniec — trzy zasady, które warto powiesić nad biurkiem

1. **Kod z planów AI to pseudokod.** Zawsze weryfikuj API z aktualną dokumentacją (dotyczy zwłaszcza `city2graph` i instalacji PyG na ROCm).
2. **Baseline jest Twoim przyjacielem.** Bardzo możliwe, że prosty XGBoost/LightGBM będzie Twoim najlepszym modelem — i to jest wynik, nie porażka.
3. **Nazywaj wyniki ostrożnie.** "Współwystępuje z", "koreluje z", "kandydat do analizy" — nie "spowoduje", "podniesie", "gwarantuje". Ta uczciwość jest w tym projekcie Twoją przewagą.